

Solution Requirements

Date	2 October 2026
Team ID	SWTID-2026-5625
Project Name	FitBuddy
Maximum Marks	

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should *do*).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	FitBuddy runs as a locally deployed web application. Users access it through the browser and provide their basic details (e.g., name, age, height, weight, fitness goal) through a simple input form to start a session. The Gemini API key is kept server-side in an environment (.env) file and is never exposed to the browser.	Medium
2	Authorization levels	Single role: User – can submit fitness details and goals, view AI-generated workout/diet guidance, and access the results page. Developer/Admin (team members with server access) – can configure the Gemini API key and model, and manage the application locally. Users cannot access server configuration.	Medium
3	External interfaces	Google Gemini API (GenAI) for generating personalised fitness, workout and nutrition suggestions. FastAPI backend serves HTML pages rendered with Jinja2 templates. No external database is required; all processing is done per request.	High
4	Transactions processing	User input is submitted from the frontend form to FastAPI routes (HTTP POST), validated, converted into a structured prompt, and sent to the Gemini model. The AI response is received, formatted, and rendered back to the user through Jinja2 templates. Errors such as API failure or invalid input return a clear message to the user.	High
5	Reporting	The system displays a personalised results page containing the AI-generated fitness plan, workout suggestions, diet/nutrition tips, and health guidance based on the user's input. Results are shown on screen in a readable, structured format.	Medium

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
6	Business rules, etc.	(a) All mandatory fields (e.g., age, height, weight, goal) must be filled and valid before submission. (b) Prompts sent to Gemini are built from the user's inputs and must stay within fitness and wellness topics. (c) Suggestions are for general guidance only and do not replace advice from a doctor or certified trainer. (d) The workflow is: Input form ® Validation ® Gemini processing ® Result display.	High
7	Compliance to laws or regulations	User details are used only to generate the response and are not shared with third parties other than the Gemini API. The application follows basic data-privacy practices (data minimisation, no unnecessary storage of personal data) and Google's Gemini API terms of use. A health disclaimer is shown with the AI results.	Medium
8	Other	Responsive and simple user interface built with HTML/CSS and Jinja2 templates; clear error and loading messages; the application can be deployed and run locally with minimal setup (Python, FastAPI, Uvicorn).	Low

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Pages and form submissions must respond quickly; AI-generated results are returned within a reasonable time depending on the Gemini API.	Pages load in < 2 seconds; AI response displayed in < 10 seconds
2	Scalability	The application is deployed locally for the current scope and handles multiple sequential user requests. The FastAPI backend is stateless and can be scaled later by deploying on a cloud server.	Supports at least 10 concurrent users locally
3	Security & Data Privacy	API key stored in environment variables; server-side input validation and sanitisation; no unnecessary storage of user data; communication with Gemini API over HTTPS.	API key never exposed to the client; HTTPS for all API calls; 0 personal records stored permanently
4	Reliability & Availability	The application should run stably during local deployment and handle failures (API timeout, invalid input, network error) gracefully without crashing.	99% uptime during demo/use; friendly error shown for 100% of handled failures
5	Usability & Accessibility	Simple, clean and user-friendly interface with clear labels, readable fonts and responsive layout for desktop and mobile browsers.	A new user completes the flow in < 2 minutes; follows WCAG 2.1 AA basics (contrast, labels)
6	Other	Maintainability: modular code structure (routes, AI logic, templates separated). Portability: runs on Windows/Linux/macOS with Python and a requirements file.	Setup in < 10 minutes using requirements.txt